

교과목 3 : 초거대언어모델(LLM)

단원 6 : 자연어-이미지 멀티모달

DAY1. 합성곱 신경망(CNN)의 구조와 연산 원리

목 차

1. 멀티모달 인공지능		3
2. CNN 구조와 연산 원리		25
3. STT (Speech-to-Text)		55

1. 멀티모달 인공지능

1. 멀티모달의 기본 개념

1.1 모달리티란 무엇인가

모달리티(Modality)란 정보가 표현되거나 인간이 정보를 받아들이는 감각적·형식적 유형을 의미한다.

대표적인 모달리티는 다음과 같다.

모달리티	데이터 예시	주요 처리 기술
텍스트	문장, 문서, 질문, 코드	자연어 처리, 언어 모델
이미지	사진, 도면, 의료 영상	컴퓨터 비전, 이미지 인코더
음성	발화, 회의 녹음, 통화	음성 인식, 음성 합성
영상	동영상, CCTV, 강의 영상	비디오 이해, 행동 인식
표·차트	표, 그래프, 대시보드	문서 이해, 구조 분석
센서	온도, 압력, 속도, 위치	시계열 분석, 이상 탐지
포인트 클라우드	LiDAR 3 차원 좌표	3 차원 객체 인식
동작·제스처	손동작, 신체 자세	포즈 추정, 행동 인식

같은 종류의 데이터라도 관측 방식이 다르면 하나의 모달리티 안에서 여러 관점으로 구분할 수 있다. 예를 들어 같은 사물을 여러 카메라 각도에서 촬영한 경우는 이미지라는 동일 모달리티에 속하지만, 여러 시점의 정보를 활용하는 **멀티뷰(Multi-view)** 데이터로 볼 수 있다.

1.2 멀티모달 인공지능의 정의

멀티모달 인공지능(Multimodal Artificial Intelligence)은 텍스트, 이미지, 음성, 영상, 센서 등 서로 다른 형태의 데이터를 함께 입력받아 이들 사이의 관계를 학습하고, 통합적으로 이해·추론·생성하는 인공지능 기술이다.

핵심은 단순히 여러 데이터를 동시에 보여주는 것이 아니다. 모델 내부에서 서로 다른 모달리티의 정보를 의미적으로 연결해야 한다.

예를 들어 다음과 같은 입력을 생각할 수 있다.

- 이미지: 도로 위에 멈춰 있는 자동차 사진
- 텍스트: "이 차량이 멈춰 있는 이유는 무엇인가?"
- 센서: 차량 속도 0km/h
- 지도 정보: 전방 교차로
- 출력: "전방 신호가 적색이므로 차량이 정지한 것으로 판단된다."

이 과정에서 모델은 이미지 속 신호등과 차량을 인식하고, 센서 속도와 지도 정보를 함께 사용하여 상황을 추론한다.

즉, 멀티모달 인공지능은 다음 과정을 수행한다.

서로 다른 데이터 입력 → 특징 추출 → 의미적 정렬 → 정보 융합 → 추론 → 결과 생성

1.3 멀티모달의 핵심 특징

1. 서로 다른 정보의 결합

텍스트는 의미를 직접 표현하는 데 강점이 있고, 이미지는 공간적 관계를 표현하는 데 유리하다. 음성은 말의 내용뿐 아니라 억양, 감정, 화자의 상태를 포함할 수 있다. 멀티모달 모델은 각 데이터의 장점을 결합하여 단일 모달리티만으로는 얻기 어려운 정보를 추론한다.

2. 상호 보완성

하나의 모달리티에서 정보가 부족하거나 불명확할 때 다른 모달리티가 이를 보완할 수 있다. 예를 들어 소음이 심한 환경에서는 음성만으로 정확한 발화를 인식하기 어렵다. 이때 화자의 입 모양을 영상으로 함께 분석하면 음성 인식 정확도를 높일 수 있다.

3. 문맥적 이해

이미지 안에 "출입 금지"라는 표지판이 있을 때, 모델은 단순히 글자만 읽는 것이 아니라 표지판의 위치, 주변 도로 상황, 차량의 진행 방향을 함께 고려할 수 있다.

4. 복합적 추론

멀티모달 인공지능은 객체 인식에 머무르지 않고, 다음과 같은 복합 질문을 처리할 수 있다.

- 그림 속 인물은 무엇을 하고 있는가?
- 차트의 수치가 증가한 원인은 무엇인가?
- 영상에서 위험한 행동이 발생했는가?
- 문서의 표와 본문 내용이 일치하는가?

2. 유니모달과 멀티모달의 차이

2.1 유니모달 인공지능

유니모달 인공지능(Unimodal AI)은 한 종류의 데이터만 입력받아 처리하는 인공지능이다.

예시는 다음과 같다.

- 텍스트만 처리하는 문서 분류 모델
- 이미지만 처리하는 객체 탐지 모델
- 음성만 처리하는 음성 인식 모델
- 센서 수치만 처리하는 이상 탐지 모델

유니모달 모델은 특정 작업에서 높은 성능을 낼 수 있지만 다른 형태의 정보를 함께 고려하지 못한다.

예를 들어 이미지 분류 모델은 사진 속 동물을 인식할 수 있지만, 사용자의 자연어 질문에 답하거나 동물의 울음소리까지 함께 분석하지는 못한다.

2.2 유니모달과 멀티모달 비교

구분	유니모달	멀티모달
입력 데이터	한 종류	두 종류 이상
예시	텍스트만 분석	텍스트와 이미지 동시 분석
장점	구조가 단순하고 학습이 비교적 쉬움	풍부한 문맥과 복합 정보 활용
단점	정보 부족과 문맥 손실 가능	학습 구조와 데이터 구축이 복잡
대표 작업	문서 분류, 이미지 분류	시각 질의응답, 영상 요약
추론 범위	단일 정보 중심	모달리티 간 관계 중심

멀티모달 모델이 항상 유니모달 모델보다 우수한 것은 아니다. 단순한 이미지 분류처럼 한 종류의 데이터만으로 충분한 작업에서는 유니모달 모델이 더 효율적일 수 있다.

3. 멀티미디어, 크로스모달, 멀티모달의 차이

3.1 멀티미디어

멀티미디어(Multimedia)는 텍스트, 이미지, 음성, 영상 등 여러 표현 형식이 하나의 콘텐츠에 포함된 것을 의미한다.

예를 들어 온라인 강의는 영상, 음성, 자막, 그림을 함께 제공한다. 그러나 해당 콘텐츠를 사용하는 시스템이 이 정보들을 의미적으로 연결하여 추론하지 않는다면 멀티모달 인공지능이라고 할 수 없다.

즉, 멀티미디어는 **정보의 표현 방식**에 가깝다.

3.2 크로스모달

크로스모달(Cross-modal)은 한 모달리티의 정보를 다른 모달리티와 연결하거나 변환하는 기술이다.

예시는 다음과 같다.

- 텍스트로 이미지를 검색
- 이미지에 맞는 설명문 검색
- 음성을 텍스트로 변환
- 텍스트를 음성으로 변환
- 문장으로 영상을 검색

크로스모달의 핵심은 서로 다른 데이터 사이의 **매핑과 변환**이다.

3.3 멀티모달

멀티모달은 여러 모달리티를 동시에 활용하여 하나의 통합된 표현을 만들고, 이를 바탕으로 학습하거나 추론하는 기술이다.

개념	핵심 의미	예시
멀티미디어	여러 미디어의 동시 제공	영상, 자막, 음성이 포함된 강의
크로스모달	모달리티 간 검색·변환	텍스트로 이미지 검색
멀티모달	여러 모달리티의 의미적 통합	사진과 질문을 함께 분석하여 답변
멀티태스크	여러 작업을 하나의 모델이 수행	분류, 요약, 번역을 함께 수행

특히 **멀티태스크(Multi-task)**와 멀티모달을 혼동하지 않아야 한다. 멀티태스크는 입력 데이터의 종류보다 모델이 수행하는 작업의 수에 초점을 둔다.

4. 멀티모달 인공지능의 등장 배경

4.1 현실 세계 정보의 복합성

인간은 한 가지 감각만으로 주변을 이해하지 않는다. 사람과 대화할 때 다음과 같은 정보를 동시에 사용한다.

- 상대방이 말한 문장
- 목소리의 크기와 억양
- 표정
- 시선
- 몸짓
- 주변 상황

따라서 현실 세계를 더 정확하게 이해하려면 인공지능도 여러 종류의 데이터를 함께 처리할 필요가 있다.

4.2 단일 모달리티의 한계

텍스트만 사용하는 경우

텍스트는 논리적 의미 전달에 강하지만 실제 공간, 색상, 위치, 표정과 같은 시각 정보를 직접 포함하지 못한다.

이미지만 사용하는 경우

이미지는 객체와 공간 관계를 보여주지만 촬영 목적, 상황의 배경, 사용자의 의도와 같은 추상적 문맥은 부족할 수 있다.

음성만 사용하는 경우

음성은 발화 내용과 억양을 포함하지만 주변 환경이나 화자의 입 모양을 확인할 수 없다. 이러한 한계를 해결하기 위해 여러 데이터의 장점을 결합하는 방식이 필요해졌다.

4.3 디지털 데이터의 다양화

스마트폰, CCTV, 웨어러블 기기, 자율주행 차량, 산업용 센서가 확산되면서 다음과 같은 복합 데이터가 대량으로 생성되기 시작했다.

- 사진과 위치 정보
- 영상과 음성
- 문서 이미지와 표
- 자동차 카메라와 LiDAR
- 건강 기록과 생체 신호
- 제조 설비 센서와 작업 지시문

데이터가 복합화됨에 따라 인공지능도 이를 통합적으로 처리해야 할 필요성이 커졌다.

4.4 딥러닝 기술의 발전

과거에는 이미지, 음성, 텍스트를 각각 별도의 알고리즘으로 처리했다. 그러나 딥러닝의 발전으로 각 모달리티를 벡터 형태로 표현하고, 동일한 신경망 구조 안에서 결합하는 것이 가능해졌다.

특히 다음 기술이 중요한 역할을 했다.

- 합성곱 신경망
- 순환 신경망
- Transformer
- Attention
- 대규모 사전학습
- 대조학습
- 생성 모델
- 확산 모델

4.5 컴퓨팅 자원의 발전

멀티모달 학습은 대규모 이미지, 텍스트, 음성, 영상을 동시에 처리해야 하므로 많은 연산량이 필요하다.

GPU와 분산 학습 기술의 발전으로 대규모 멀티모달 데이터 학습이 가능해졌다.

주요 기반 기술은 다음과 같다.

- GPU 병렬 처리
- 다중 GPU 학습
- 데이터 병렬화
- 모델 병렬화
- 혼합 정밀도 학습
- 클라우드 컴퓨팅
- 고속 스토리지와 네트워크

5. 멀티모달 인공지능의 기본 구조

멀티모달 모델의 일반적인 구조는 다음과 같다.



5.1 전처리

각 모달리티는 데이터 형식이 다르기 때문에 서로 다른 전처리 과정이 필요하다.

텍스트 전처리

- 문장 정제
- 토큰화
- 특수 토큰 추가
- 토큰 ID 변환
- 패딩과 길이 조정

이미지 전처리

- 이미지 크기 조정
- 픽셀 정규화
- 패치 분할
- 데이터 증강

음성 전처리

- 잡음 제거
- 샘플링 주파수 통일
- 음성 구간 검출
- 스펙트로그램 변환

영상 전처리

- 프레임 추출
- 시간 간격 정렬
- 해상도 조정
- 장면 분할

센서 데이터 전처리

- 결측치 처리
- 이상값 제거
- 시계열 정렬
- 단위 표준화

5.2 모달리티별 인코더

인코더는 각 입력 데이터를 모델이 처리할 수 있는 특징 벡터로 변환한다.

텍스트 인코더

문장을 토큰 단위로 나누고 문맥적 의미를 벡터로 표현한다.

“강아지가 공원에서 달린다”

- 토큰 분리
- 각 토큰의 임베딩 생성
- 문장 문맥 표현 생성

이미지 인코더

이미지를 여러 패치 또는 특징 영역으로 나누어 객체, 색상, 형태, 위치 정보를 추출한다.

이미지

- 패치 분할
- 패치 임베딩
- 객체 및 공간 관계 표현

음성 인코더

음성 파형에서 발음, 억양, 리듬, 화자 특성 등을 추출한다.

영상 인코더

영상의 각 프레임에서 시각 특징을 추출하고 프레임 사이의 시간적 변화를 학습한다.

5.3 임베딩

임베딩(Embedding)은 텍스트, 이미지, 음성 등 서로 다른 데이터를 숫자로 구성된 고차원 벡터로 표현하는 기술이다.

예를 들어 다음 데이터는 형식은 다르지만 의미는 비슷하다.

- 텍스트: “고양이가 소파 위에 있다.”
- 이미지: 소파 위에 앉아 있는 고양이 사진

학습이 잘 이루어진 멀티모달 임베딩 공간에서는 두 데이터의 벡터가 가까운 위치에 배치된다.

텍스트 벡터 —
└→ 의미가 비슷하면 벡터 거리도 가까움
이미지 벡터 —

이러한 공통 임베딩 공간은 이미지 검색, 텍스트 기반 영상 검색, 이미지-문장 매칭 등에 활용된다.

6. 모달리티 정렬

6.1 정렬의 의미

모달리티 정렬(Modality Alignment)은 서로 다른 데이터가 의미상 어떤 관계에 있는지 대응시키는 과정이다.

예를 들어 이미지 안의 특정 영역과 문장의 특정 단어를 연결할 수 있다.

이미지 영역		문장 토큰
강아지 영역	↔	"강아지"
공 영역	↔	"공"
잔디 영역	↔	"공원"
달리는 자세	↔	"달린다"

정렬이 정확하지 않으면 모델은 이미지에 없는 객체를 있다고 판단하거나, 질문과 관계없는 영역에 집중할 수 있다.

6.2 공간적 정렬

이미지나 문서에서 객체의 위치와 텍스트의 의미를 연결한다.

예를 들어 영수증에서 다음 관계를 학습한다.

- "합계"라는 글자
- 오른쪽에 있는 금액
- 표의 마지막 행
- 결제 금액이라는 의미

6.3 시간적 정렬

영상과 음성처럼 시간에 따라 변화하는 데이터를 같은 시점에 맞추는 과정이다.

예를 들어 영상 속 화자의 입 모양과 음성 신호가 시간적으로 일치해야 한다.

시간적 정렬이 잘못되면 다음 문제가 발생할 수 있다.

- 자막과 음성이 맞지 않음
- 행동과 설명이 다른 시점에 연결됨
- 화자와 발화 내용이 잘못 연결됨

7. 모달리티 융합 기술

융합(Fusion)은 각 모달리티에서 추출된 특징을 결합하는 과정이다.

7.1 초기 융합

입력 데이터를 처리하는 초기 단계에서 특징을 결합한다.

```
텍스트 특징 ─┐  
                └→ 하나의 결합 벡터 → 모델 입력  
이미지 특징 ─┐
```

장점

- 모달리티 사이의 세밀한 관계를 조기에 학습할 수 있다.
- 낮은 수준의 특징을 직접 결합할 수 있다.

단점

- 데이터의 크기와 구조가 다르면 결합하기 어렵다.
- 특정 모달리티의 잡음이 전체 학습에 영향을 줄 수 있다.

7.2 후기 융합

각 모달리티를 독립적으로 처리한 뒤 최종 예측 결과를 결합한다.

```
이미지 모델 → 예측 A ─┐  
                        └→ 최종 예측  
음성 모델   → 예측 B ─┐
```

장점

- 기존 유니모달 모델을 재사용하기 쉽다.
- 특정 모달리티가 누락되어도 비교적 대응하기 쉽다.

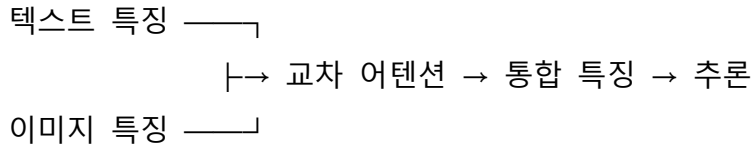
단점

- 모달리티 사이의 세밀한 상호작용을 충분히 학습하기 어렵다.

7.3 중간 융합

각 모달리티의 중간 특징을 추출한 후 모델 내부에서 반복적으로 상호작용하게 한다.

현대 멀티모달 모델에서 널리 사용하는 방식이다.



7.4 융합 방식 비교

방식	결합 시점	장점	단점
초기 융합	입력 또는 낮은 특징 단계	세밀한 상호작용	입력 구조 통일이 어려움
중간 융합	모델 내부	표현력과 유연성이 높음	구조와 학습이 복잡
후기 융합	최종 출력 단계	구현과 확장이 쉬움	깊은 상호작용이 부족

8. Attention 과 Transformer

8.1 Attention

Attention 은 입력 데이터의 모든 부분을 동일하게 처리하지 않고, 현재 작업에 중요한 부분에 더 큰 가중치를 부여하는 기술이다.

예를 들어 이미지와 함께 다음 질문이 주어졌다고 하자.

“빨간 옷을 입은 사람이 들고 있는 물건은 무엇인가?”

모델은 질문에 따라 다음 정보에 집중해야 한다.

1. 이미지에서 빨간 옷을 입은 사람 찾기
2. 해당 사람의 손 위치 찾기
3. 손에 있는 객체 확인
4. 객체 이름 생성

이 과정에서 질문의 단어와 이미지 영역 사이의 관련도를 계산하는 것이 중요하다.

8.2 Self-Attention

동일한 모달리티 내부의 관계를 학습한다.

- 텍스트 단어와 단어 사이의 관계
- 이미지 패치와 패치 사이의 관계
- 영상 프레임과 프레임 사이의 관계

8.3 Cross-Attention

서로 다른 모달리티 사이의 관계를 학습한다.

예를 들어 텍스트 토큰이 이미지의 어느 영역과 관련되는지 계산한다.

질문의 “자동차” 토큰



이미지에서 자동차가 있는 영역에 높은 가중치 부여

Cross-Attention 은 시각 질의응답, 이미지 캡셔닝, 문서 이해 등에서 중요한 기술이다.

8.4 Transformer

Transformer 는 Attention 을 중심으로 구성된 신경망 구조이다.

멀티모달 환경에서는 텍스트 토큰, 이미지 패치, 음성 구간, 영상 프레임을 모두 일종의 토큰으로 변환한 뒤 관계를 학습할 수 있다.

Transformer 가 멀티모달 처리에 적합한 이유는 다음과 같다.

- 장거리 관계 학습이 가능하다.
- 병렬 처리가 가능하다.
- 서로 다른 종류의 토큰을 통합할 수 있다.
- 대규모 사전학습에 적합하다.
- Cross-Attention 을 통한 모달리티 연결이 가능하다.

9. 멀티모달 학습 기술

9.1 지도 학습

입력 데이터와 정답을 함께 제공하여 학습하는 방식이다.

예시:

입력: 이미지 + 질문
정답: "파란색"

지도학습은 명확한 작업에 효과적이지만 정답 라벨을 사람이 직접 만들어야 하므로 비용이 크다.

9.2 자기지도 학습

데이터 자체에서 학습 신호를 만드는 방식이다.

예를 들어 이미지와 설명문이 함께 있는 데이터를 사용하여 두 데이터가 서로 일치하는지 학습할 수 있다.

- 이미지의 일부를 가리고 복원
- 문장의 일부 단어를 가리고 예측
- 영상의 다음 프레임 예측
- 이미지와 문장이 같은 내용을 설명하는지 판단

대규모 멀티모달 모델은 자기지도학습을 통해 방대한 비라벨 데이터를 활용한다.

9.3 대조 학습

대조 학습(Contrastive Learning)은 의미가 관련된 데이터 쌍은 가깝게, 관련 없는 데이터 쌍은 멀게 배치하도록 학습하는 방식이다.

예를 들어 다음은 긍정 쌍이다.

강아지 사진 ↔ "잔디밭에서 달리는 강아지"

다음은 부정 쌍이다.

강아지 사진 ↔ "바다 위를 항해하는 선박"

대조학습은 텍스트 기반 이미지 검색과 이미지 기반 문장 검색에서 핵심적인 역할을 한다.

9.4 마스킹 학습

입력의 일부를 숨기고 복원하게 하는 방식이다.

- 문장 일부 토큰 숨기기
- 이미지 일부 패치 숨기기
- 영상 일부 프레임 숨기기
- 음성 일부 구간 숨기기

이를 통해 모델은 데이터 내부 구조와 문맥을 학습한다.

9.5 생성 학습

입력 모달리티를 기반으로 다른 형태의 데이터를 생성한다.

예시는 다음과 같다.

- 이미지 → 설명문
- 텍스트 → 이미지
- 텍스트 → 음성
- 음성 → 텍스트
- 텍스트 → 영상
- 영상 → 요약문

10. 모달리티 변환 기술

10.1 텍스트에서 이미지로

텍스트 설명을 바탕으로 이미지를 생성하거나 기존 이미지를 편집한다.

활용 분야:

- 광고 시안 제작
- 제품 디자인
- 게임 캐릭터 제작
- 교육 자료 제작
- 인테리어 시뮬레이션

핵심 기술:

- 텍스트 인코더
- 이미지 생성 모델
- 확산 모델
- 잠재 공간
- 조건부 생성
- 이미지 디코더

10.2 텍스트에서 음성으로

문장을 자연스러운 음성으로 변환한다.

활용 분야:

- 내레이션
- 전자책 낭독
- 시각장애인 접근성 지원
- 안내 방송
- 음성 비서

핵심 기술:

- 문장 분석
- 음소 변환
- 운율 예측
- 음향 특징 생성
- 보코더

10.3 음성에서 텍스트로

사용자의 음성을 문장으로 변환한다.

활용 분야:

- 회의록 작성
- 자막 생성
- 음성 명령
- 상담 기록
- 강의 내용 기록

주요 처리 과정:

음성 입력

→ 잡음 제거

→ 음성 구간 검출

→ 음향 특징 추출

→ 발음 단위 인식

→ 언어 모델 보정

→ 텍스트 출력

10.4 텍스트에서 영상으로

장면 설명이나 스토리보드를 입력하여 영상을 생성한다.

활용 분야:

- 광고 영상
- 교육용 애니메이션
- 콘텐츠 시각화
- 시제품 영상
- 영화 사전 시각화

한계:

- 높은 연산 비용
- 프레임 간 일관성 문제
- 긴 영상에서 객체 유지 문제
- 생성 지연
- 저작권 및 인물 권리 문제

10.5 이미지에서 이미지로

원본 이미지를 조건으로 새로운 이미지를 생성하거나 편집한다.

대표 작업:

- 인페인팅
- 배경 제거
- 객체 삭제
- 스타일 변환

- 이미지 복원
- 해상도 향상
- 의미적 분할

10.6 비디오에서 비디오로

원본 영상의 주요 특징을 유지하면서 영상을 변환한다.

대표 작업:

- 영상 요약
- 하이라이트 생성
- 스타일 변환
- 특정 객체 추적
- 장면 경계 검출
- 저해상도 영상 복원

11. 주요 멀티모달 작업

11.1 시각 질의응답

시각 질의응답(Visual Question Answering, VQA)은 이미지와 자연어 질문을 함께 입력받아 답을 생성하는 작업이다.

예시:

입력 이미지: 책상 위에 노트북과 컵이 있는 사진

질문: "컵은 노트북의 어느 쪽에 있는가?"

출력: "노트북의 오른쪽에 있다."

필요 기술:

- 객체 인식
- 위치 관계 이해
- 질문 의도 분석
- 이미지-텍스트 정렬
- 자연어 답변 생성

11.2 문서 시각 질의응답

문서 시각 질의응답(Document VQA)은 문서 이미지의 문자, 표, 레이아웃을 함께 분석하여 질문에 답하는 작업이다.

단순한 문자 인식만으로는 충분하지 않다. 문서에서 각 정보가 어느 위치와 구조에 속하는지 이해해야 한다.

예를 들어 계약서에서 다음을 추출할 수 있다.

- 계약 당사자
- 계약 기간
- 계약 금액
- 해지 조건
- 서명 여부

필요 기술:

- 광학 문자 인식
- 레이아웃 분석
- 표 구조 인식
- 키-값 추출
- 문서 임베딩
- 질의응답 모델

11.3 차트 질의응답

차트 질의응답(Chart QA)은 그래프나 차트 이미지와 질문을 함께 분석하여 답을 생성한다.

예시:

- 어느 연도의 매출이 가장 높은가?
- 2024 년 대비 2025 년 매출은 얼마나 증가했는가?
- 전체적인 추세는 증가인가 감소인가?
- 가장 큰 변동이 발생한 구간은 어디인가?

차트 분석에서는 단순 이미지 인식뿐 아니라 축, 범례, 단위, 수치 관계를 이해해야 한다.

11.4 비디오 질의응답

비디오 질의응답(Video QA)은 영상과 질문을 함께 입력받아 특정 장면, 행동, 사건을 설명한다. 영상은 이미지와 달리 시간 순서가 중요하다.

모델은 다음을 이해해야 한다.

- 먼저 발생한 사건
- 이후 발생한 행동
- 행동의 원인과 결과
- 동일 객체의 시간적 변화
- 장면 전환

11.5 이미지 캡셔닝

이미지 캡셔닝(Image Captioning)은 이미지의 내용을 자연어 문장으로 설명하는 작업이다.

입력: 자전거를 타고 공원을 지나가는 사람의 사진

출력: "한 사람이 공원 산책로에서 자전거를 타고 있다."

좋은 캡션을 생성하려면 객체 이름뿐 아니라 행동, 위치, 관계, 배경을 함께 이해해야 한다.

11.6 음성 질의응답

음성과 질문을 함께 분석하거나, 음성으로 입력된 질문의 의미를 이해하여 답변하는 작업이다.

회의 분석에서는 다음과 같은 질문을 처리할 수 있다.

- 회의에서 최종 결정된 사항은 무엇인가?
- 특정 발언자는 어떤 의견을 제시했는가?
- 일정과 담당자는 누구인가?
- 해결되지 않은 문제가 무엇인가?

11.7 오디오-비주얼 음성 인식

오디오-비주얼 음성 인식(Audio-Visual Speech Recognition)은 음성과 화자의 입 모양을 함께 분석하는 기술이다.

활용 환경:

- 소음이 심한 공장
- 차량 내부
- 여러 사람이 동시에 말하는 회의실
- 원거리 영상 통화

영상 정보가 음성 인식을 보완하므로 소음 환경에서 정확도를 높일 수 있다.

11.8 비전-언어 내비게이션

비전-언어 내비게이션(Vision-Language Navigation)은 자연어 지시와 시각 환경을 함께 사용하여 로봇이나 에이전트의 이동 경로를 결정하는 기술이다.

예시 지시:

“복도를 따라가다가 두 번째 문에서 오른쪽으로 들어가 책상 앞에 멈추시오.”

모델은 카메라 영상을 분석하면서 ‘복도’, ‘두 번째 문’, ‘오른쪽’, ‘책상’의 의미를 실제 공간과 연결해야 한다.

11.9 멀티모달 요약

멀티모달 요약은 영상, 음성, 자막, 문서 등 여러 데이터를 분석하여 핵심 내용을 압축하는 작업이다.

예를 들어 강의 영상 요약에서는 다음 정보가 결합된다.

- 강사의 음성
- 자동 생성 자막
- 강의 화면
- 슬라이드 텍스트
- 판서 내용
- 화면 전환 시점

결과는 요약문, 핵심 장면, 주요 키워드, 시간대별 목차 등의 형태로 출력될 수 있다.

12. 입력 모달리티 조합과 대표 활용

입력 조합	대표 출력	활용 사례	기술적 고려 사항
텍스트+이미지	답변, 설명, 분류	VQA, 캡셔닝, 이미지 검색	객체와 단어 정렬
텍스트+비디오	설명, 요약	영상 질의응답, 하이라이트	시간 정보 처리
텍스트+오디오	전사, 요약	회의 요약, 음성 질의응답	잡음 및 화자 분리
이미지+레이아웃	구조화 데이터	계약서, 영수증, 문서 분석	OCR 과 레이아웃 분석
비디오+오디오	장면 설명, 분류	행동 인식, 영상 이해	시간 동기화
포인트 클라우드+텍스트	경로, 설명	로봇, 자율주행	좌표계와 센서 보정
센서+텍스트	경고, 보고서	설비 이상 탐지	단위·시간 정렬
차트+텍스트	근거 기반 답변	데이터 분석, 보고서 작성	수치와 축 정보 추출

2. CNN 구조와 연산 원리

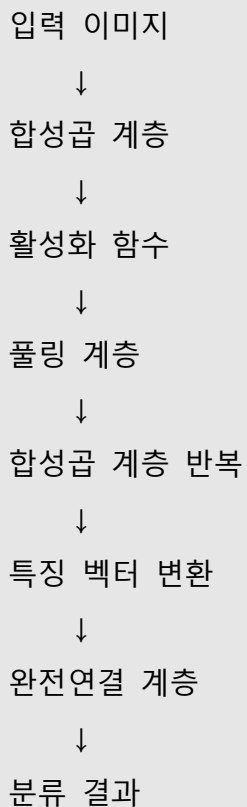
1. CNN 의 개념

1.1 CNN 이란

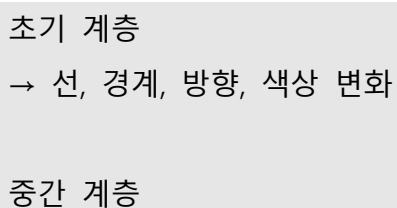
CNN 은 Convolutional Neural Network 의 약자로, 한국어로는 합성곱 신경망이라고 한다.

CNN 은 이미지와 같이 격자 구조를 가지는 데이터를 처리하는 데 적합한 인공신경망이다. 이미지의 가로, 세로, 채널 구조를 유지하면서 주변 픽셀 사이의 관계를 분석하고, 분류나 탐지에 필요한 특징을 자동으로 학습한다.

일반적인 CNN 의 처리 과정은 다음과 같다.



CNN 은 이미지에서 특징을 계층적으로 학습한다.



→ 곡선, 질감, 눈, 코, 바퀴

깊은 계층

→ 얼굴, 차량 부품, 동물의 몸통

출력 계층

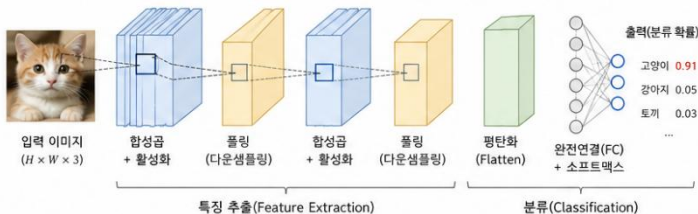
→ 고양이, 자동차, 사람 등의 클래스

CNN의 중요한 특징은 사람이 이미지의 특징을 직접 정의하지 않아도 학습 과정에서 필요한 특징을 자동으로 찾는다라는 점이다.

CNN(Convolutional Neural Network)의 구조와 연산 원리

CNN은 이미지와 같은 격자 형태의 데이터를 효과적으로 처리하기 위해 제안된 딥러닝 모델이다.
합성곱(Convolution) 연산을 핵심으로 하며, 공간적 특징을 자동으로 학습한다.

1. CNN의 전체 구조



- 합성곱층(Convolution Layer)과 풀링층(Pooling Layer)을 여러 번 쌓아 특징을 추출하고, 추출된 특징을 완전연결층(FC)을 통해 분류한다.

2. 합성곱(Convolution) 연산

- 입력 특징맵 위에 작은 필터(커널)를 슬라이드하며 원소곱 후 합을 구한다.

입력 특징맵 (예시)

1	0	2	1	0
0	1	1	0	2
2	1	0	1	1
0	2	1	0	1
1	0	1	2	1

필터(커널)

1	0	-1
1	0	-1
1	0	-1

출력 특징맵

2	-1	-2
3	0	-3
2	-1	-1

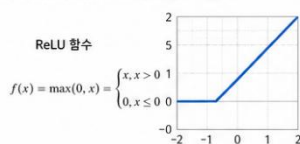
*

=

- 각 위치에서의 계산 예 (좌상단 출력값)
 $1 \times 1 + 0 \times 0 + 2 \times (-1) + 0 \times 1 + 1 \times 0 + 1 \times (-1) + 2 \times 1 + 1 \times 0 + 0 \times (-1) = 2$
- 스트라이드(stride): 필터가 이동하는 간격 (위 예시: 1)
- 패딩(padding): 입력의 가장자리에 0을 추가하여 출력 크기 조절

3. 활성화 함수

합성곱의 결과에 비선형성을 추가하여 복잡한 패턴을 학습할 수 있게 한다.
주로 ReLU(Rectified Linear Unit)를 사용한다.



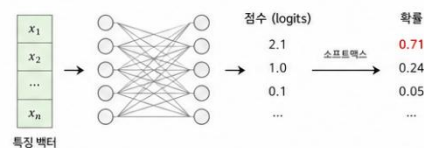
4. 풀링(Pooling) 연산

특징맵의 크기를 줄여 계산량을 감소시키고, 위치 변화에 대한 강건성을 높인다.



5. 분류(완전연결층 + 소프트맥스)

평탄화된 특징 벡터를 완전연결층에 통과시켜 클래스별 점수를 구하고, 소프트맥스(softmax)로 확률로 변환한다.



6. 학습 원리

정답 레이블과 모델의 예측 확률을 비교하여 손실함수(예: 교차엔트로피)를 계산하고, 역전파(Backpropagation)와 경사하강법(Gradient Descent)을 통해 모든 가중치(필터, FC 가중치 등)를 업데이트한다.

핵심 요약

- CNN은 합성곱과 풀링을 통해 이미지의 공간적 특징을 계층적으로 학습한다.
- 파라미터 공유(필터의 모든 위치에 동일 적용)로 효율적이며, 번역(위치) 변화에 강건하다.

2. 일반 신경망으로 이미지를 처리할 때의 문제

2.1 이미지 평탄화 문제

28×28 크기의 흑백 이미지는 총 784 개의 픽셀로 구성된다.

$$28 \times 28 = 784$$

일반적인 완전연결 신경망에 이미지를 입력하려면 2 차원 이미지를 1 차원 벡터로 변환해야 한다.

28×28 이미지 ↓ Flatten 784 개 숫자로 구성된 벡터
--

이 과정을 평탄화(Flatten)라고 한다.

평탄화를 수행하면 이미지에 존재하던 가로와 세로의 공간적 관계가 약해진다. 원래 서로 인접한 픽셀도 신경망에서는 단순히 벡터의 서로 다른 위치로 처리된다.

2.2 파라미터 수 증가

224×224 크기의 RGB 이미지는 다음과 같은 입력값을 가진다.

$$224 \times 224 \times 3 = 150,528$$

이 입력을 1,000 개의 뉴런과 완전연결하면 필요한 가중치 수는 다음과 같다.

$$150,528 \times 1,000 = 150,528,000$$

편향을 제외하더라도 약 1 억 5 천만 개의 파라미터가 필요하다.

이러한 구조에는 다음 문제가 발생한다.

- 메모리 사용량이 커진다.
- 학습 시간이 길어진다.
- 과적합 가능성이 증가한다.
- 이미지의 공간 구조를 충분히 활용하지 못한다.
- 객체 위치가 조금만 달라져도 다른 입력 패턴으로 처리된다.

CNN 은 지역적 연결과 가중치 공유를 이용하여 이러한 문제를 해결한다.

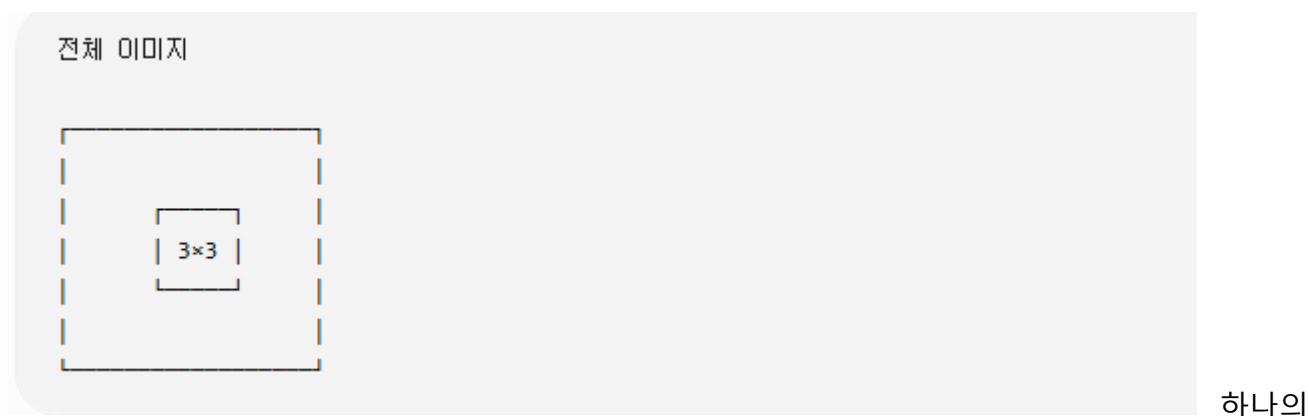
3. CNN의 핵심 원리

CNN은 다음 세 가지 원리를 중심으로 동작한다.

3.1 지역적 연결

CNN의 뉴런은 이전 계층의 모든 값과 연결되지 않는다. 대신 입력 데이터의 작은 영역만 관찰한다.

예를 들어 3×3 커널은 한 번에 9개의 픽셀을 처리한다.

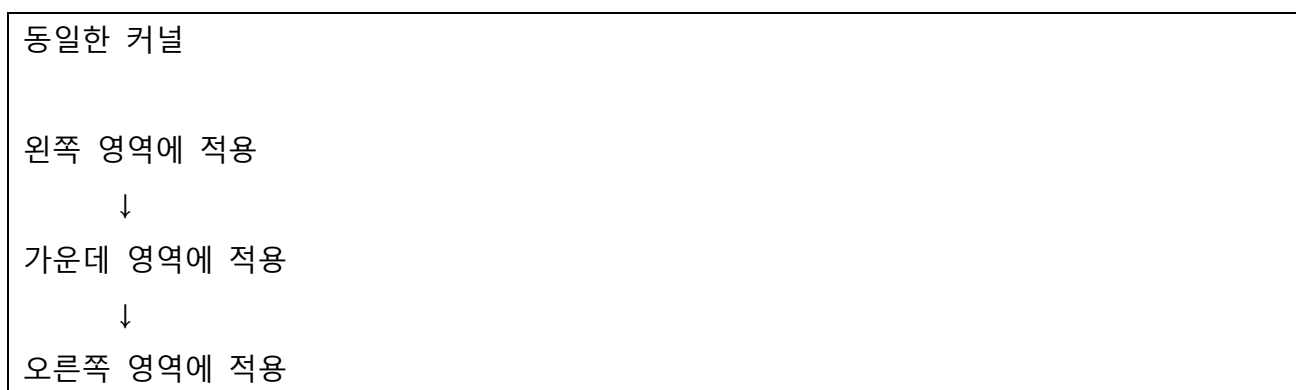


뉴런이 입력 이미지에서 영향을 받는 영역을 수용 영역(Receptive Field)이라고 한다.

이미지에서 가까운 픽셀끼리는 서로 관련성이 높기 때문에 작은 지역을 중심으로 분석하는 방식이 효과적이다.

3.2 가중치 공유

CNN은 하나의 커널을 이미지의 모든 위치에 반복적으로 적용한다.



이 방식을 사용하면 동일한 특징이 이미지의 어느 위치에 나타나더라도 같은 커널로 검출할 수 있다.

가중치 공유의 장점은 다음과 같다.

- 파라미터 수 감소
- 계산량 감소
- 위치 변화에 대한 대응력 향상
- 동일한 특징의 반복 검출

3.3 계층적 특징 학습

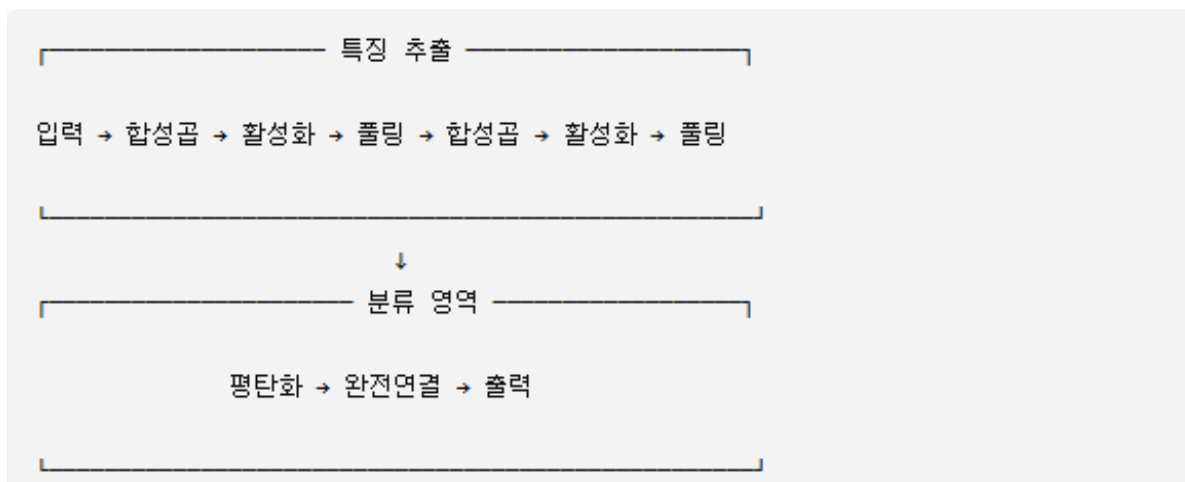
CNN 은 여러 합성곱 계층을 통과하면서 단순한 특징을 조합하여 복잡한 특징을 만든다.



초기 계층에서는 선이나 모서리와 같은 저수준 특징을 학습하고, 깊은 계층에서는 얼굴이나 자동차처럼 의미가 있는 고수준 특징을 학습한다.

4. CNN 의 전체 구조

일반적인 CNN 은 크게 특징 추출 영역과 분류 영역으로 나눌 수 있다.



각 계층의 역할은 다음과 같다.

계층	역할
입력 계층	이미지 데이터를 모델에 전달
합성곱 계층	지역적인 시각 특징 추출
활성화 함수	비선형성 추가
풀링 계층	특성 맵 크기 축소
평탄화 계층	다차원 특성 맵을 1 차원으로 변환
완전연결 계층	추출된 특징을 종합
출력 계층	클래스별 점수 또는 확률 출력

5. 이미지 데이터의 표현

5.1 흑백 이미지

흑백 이미지는 하나의 채널을 가진다.

$$H \times W \times 1$$

28×28 크기의 흑백 이미지는 다음 형태를 가진다.

$$28 \times 28 \times 1$$

각 픽셀은 일반적으로 0 부터 255 사이의 값으로 표현된다.

0 : 검은색
255 : 흰색
중간값: 회색

5.2 RGB 이미지

컬러 이미지는 일반적으로 세 개의 채널로 구성된다.

- R: Red
- G: Green
- B: Blue

이미지 형태는 다음과 같다.

$$H \times W \times 3$$

예를 들어 32×32 컬러 이미지는 다음 형태이다.

$$32 \times 32 \times 3$$

빨간색 픽셀은 다음처럼 표현할 수 있다.

$$R = 255$$

$$G = 0$$

$$B = 0$$

5.3 PyTorch 이미지 텐서 구조

PyTorch에서는 일반적으로 이미지 텐서를 다음 순서로 표현한다.

$$N \times C \times H \times W$$

각

기호의 의미는 다음과 같다.

- (N): 배치 크기
- (C): 채널 수
- (H): 이미지 높이
- (W): 이미지 너비

예를 들어 32 장의 RGB 224×224 이미지는 다음 형태이다.

$$32 \times 3 \times 224 \times 224$$

6. 합성곱 계층

6.1 합성곱 연산의 개념

합성곱 연산은 작은 크기의 필터를 입력 이미지 위에서 이동시키면서 특징을 계산하는 연산이다.

필터는 다음과 같은 이름으로도 부른다.

- 커널
- 합성곱 필터
- 특징 검출기

기본 과정은 다음과 같다.

1. 입력 이미지에서 작은 영역을 선택한다.
2. 선택 영역과 커널의 같은 위치 값을 곱한다.
3. 곱셈 결과를 모두 더한다.
4. 필요하면 편향을 더한다.
5. 결과를 출력 특성 맵에 저장한다.
6. 커널을 다음 위치로 이동한다.

입력이 (X), 커널이 (K), 편향이 (b)일 때 출력은 다음과 같이 표현할 수 있다.

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n) + b$$

6.2 합성곱 계산 예제

다음과 같은 4×4 입력 행렬이 있다고 하자.

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 2 & 1 & 0 \end{bmatrix}$$

2×2 커널은 다음과 같다.

$$K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

커널을 입력의 왼쪽 위에 놓으면 선택되는 영역은 다음과 같다.

$$\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$$

같은 위치의 값끼리 곱한 후 모두 더한다.

$$\begin{aligned} & (1 \times 1) + (2 \times 0) + (3 \times 0) + (1 \times -1) \\ & = 1 + 0 + 0 - 1 = 0 \end{aligned}$$

따라서 출력 특성 맵의 첫 번째 값은 0이다.

커널을 오른쪽으로 한 칸 이동하면 다음 영역이 선택된다.

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

계산 결과는 다음과 같다.

$$(2 \times 1) + (0 \times 0) + (1 \times 0) + (2 \times -1) = 0$$

이 과정을 입력 전체에 반복하여 특성 맵을 생성한다.

6.3 합성곱과 교차상관

수학적인 의미의 합성곱은 커널을 상하좌우로 뒤집은 후 계산한다.

하지만 대부분의 딥러닝 라이브러리는 커널을 뒤집지 않고 그대로 이동시킨다. 따라서 실제 연산은 엄밀하게는 교차상관(Cross-correlation)에 가깝다.

그러나 커널 값이 학습 과정에서 자동으로 결정되기 때문에 딥러닝에서는 관례적으로 이를 합성곱이라고 부른다.

7. 커널과 특성 맵

7.1 커널의 역할

커널은 이미지에서 특정한 패턴을 검출하는 작은 가중치 행렬이다.

예를 들어 다음 커널은 세로 방향의 경계를 강조할 수 있다.

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

가로 방향의 경계를 찾는 커널은 다음과 같이 구성할 수 있다.

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

전통적인 영상 처리에서는 사람이 커널 값을 직접 설계했다. CNN에서는 커널 값이 학습 과정에서 자동으로 결정된다.

7.2 특성 맵

커널 하나를 입력 전체에 적용한 결과를 특성 맵(Feature Map)이라고 한다.

입력 이미지

- └─ 커널 1 → 특성 맵 1
- └─ 커널 2 → 특성 맵 2
- └─ 커널 3 → 특성 맵 3
- └─ 커널 4 → 특성 맵 4

커널마다 서로 다른 특징을 학습한다.

- 세로선
- 가로선
- 대각선
- 색상 변화
- 질감
- 객체 일부

필터 하나는 출력 채널 하나를 생성한다.

8. 다중 채널 합성곱

8.1 RGB 이미지의 커널 구조

RGB 이미지는 입력 채널이 3 개이므로 커널도 깊이 3 을 가져야 한다.

입력 이미지: $32 \times 32 \times 3$

커널 크기 : $3 \times 3 \times 3$

하나의 커널은 R, G, B 채널별 합성곱 결과를 모두 더한다.

$$Y = X_R * K_R + X_G * K_G + X_B * K_B + b$$

하나의 $3 \times 3 \times 3$ 커널은 하나의 출력 특성 맵을 생성한다.

8.2 여러 개의 필터

출력 채널 수는 필터 수에 의해 결정된다.

입력 크기: $32 \times 32 \times 3$

필터 크기: $3 \times 3 \times 3$

필터 수 : 64 개

출력 크기: $32 \times 32 \times 64$

64 개의 필터는 서로 다른 특징을 추출하여 64 개의 특성 맵을 생성한다.

9. 스트라이드

9.1 스트라이드의 의미

스트라이드(Stride)는 커널이 한 번에 이동하는 칸 수이다.

스트라이드가 1 이면 한 칸씩 이동한다.

1 번 위치 → 2 번 위치 → 3 번 위치 → 4 번 위치

스트라이드가 2 이면 두 칸씩 이동한다.

1 번 위치 → 3 번 위치 → 5 번 위치

스트라이드가 커질수록 커널이 방문하는 위치가 감소하므로 출력 특성 맵의 크기도 작아진다.

9.2 출력 크기 계산

입력 크기를 (N), 커널 크기를 (F), 패딩을 (P), 스트라이드를 (S)라고 하면 출력 크기는 다음과 같다.

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

예를 들어 다음 조건을 사용한다고 하자.

입력 크기 $N = 5$

커널 크기 $F = 3$

패딩 $P = 0$

스트라이드 $S = 1$

출력 크기는 다음과 같다.

$$O = \frac{5 - 3}{1} + 1 = 3$$

따라서 5×5 입력은 3×3 출력으로 변환한다.

스트라이드가 2 라면 다음과 같다.

$$O = \left\lfloor \frac{5 - 3}{2} \right\rfloor + 1 = 2$$

따라서 출력 크기는 2×2 가 된다.

10. 패딩

10.1 패딩의 의미

패딩(Padding)은 입력 데이터의 가장자리에 값을 추가하는 것이다.

가장 일반적인 방법은 가장자리에 0을 추가하는 제로 패딩이다.

원본 입력이 다음과 같다고 하자.

```
1 2 3
4 5 6
7 8 9
```

패딩을 1만큼 적용하면 다음과 같이 된다.

```
0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0
```

10.2 패딩을 사용하는 이유

패딩의 주요 목적은 다음과 같다.

- 합성곱 후 이미지 크기가 빠르게 감소하는 것을 방지한다.
- 이미지 가장자리 정보를 충분히 활용한다.
- 입력과 출력의 공간 크기를 동일하게 유지할 수 있다.

10.3 Valid Padding

Valid Padding은 패딩을 사용하지 않는 방식이다.

$$P = 0$$

출력의 가로와 세로 크기는 입력보다 작아진다.

10.4 Same Padding

Same Padding 은 출력 크기가 입력 크기와 같도록 패딩하는 방식이다.

3×3 커널, 스트라이드 1, 패딩 1 을 사용하면 다음과 같다.

입력: 32×32
커널: 3×3
스트라이드: 1
패딩: 1
출력: 32×32

계산식은 다음과 같다.

$$\frac{32 - 3 + 2(1)}{1} + 1 = 32$$

11. 합성곱 계층의 파라미터 수

커널 높이를 (K_H), 커널 너비를 (K_W), 입력 채널 수를 (C_{in}), 출력 채널 수를 (C_{out})이라고 하자.

편향을 사용하는 합성곱 계층의 파라미터 수는 다음과 같다.

$$\text{파라미터 수} = (K_H \times K_W \times C_{in} + 1) \times C_{out}$$

여기서 1 은 필터별 편향을 의미한다.

예를 들어 다음 합성곱 계층을 생각해 보자.

커널 크기: 3×3
입력 채널: 3
출력 채널: 64

파라미터 수는 다음과 같다.

$$\begin{aligned}
 & (3 \times 3 \times 3 + 1) \times 64 \\
 & = (27 + 1) \times 64 \\
 & = 1,792
 \end{aligned}$$

입력 이미지 크기가 32×32 이든 224×224 이든 커널 크기와 채널 수가 같으면 파라미터 수는 동일하다.

이는 동일한 필터를 입력의 모든 위치에 반복 적용하는 가중치 공유 때문이다.

12. 활성화 함수

12.1 활성화 함수의 필요성

합성곱 연산은 선형 연산이다. 선형 연산만 여러 번 적용하면 전체 네트워크도 하나의 선형 변환과 같은 결과를 낸다.

복잡한 이미지 패턴을 학습하려면 비선형 변환이 필요하다.

CNN에서는 주로 ReLU를 사용한다.

$$\text{ReLU}(x) = \max(0, x)$$

동작 예시는 다음과 같다.

입력	출력
-3	0
-1	0
0	0
2	2
5	5

12.2 ReLU의 장점

- 계산이 간단하다.
- 양수 영역에서 기울기가 유지된다.
- 깊은 신경망 학습에 적합하다.
- 음수 값을 0으로 만들어 희소한 표현을 생성한다.

12.3 ReLU 의 한계

입력이 계속 음수이면 출력과 기울기가 모두 0 이 된다. 해당 뉴런의 가중치가 더 이상 제대로 학습되지 않을 수 있다.

이를 Dying ReLU 문제라고 한다.

대안으로 다음 활성화 함수를 사용할 수 있다.

- Leaky ReLU
- ELU
- GELU
- SiLU

13. 풀링 계층

13.1 풀링의 역할

풀링은 특성 맵의 가로와 세로 크기를 줄이는 연산이다.

주요 목적은 다음과 같다.

- 계산량 감소
- 메모리 사용량 감소
- 과적합 완화
- 작은 위치 변화에 대한 민감도 감소
- 중요한 특징 유지

풀링은 일반적으로 채널 수를 변경하지 않는다.

입력: $28 \times 28 \times 32$

2×2 풀링 적용

출력: $14 \times 14 \times 32$

13.2 최대 풀링

Max Pooling 은 지정된 영역에서 가장 큰 값을 선택한다.

다음 특성 맵에 2×2 최대 풀링을 스트라이드 2 로 적용한다.

$$\begin{bmatrix} 1 & 3 & 2 & 0 \\ 4 & 6 & 1 & 2 \\ 2 & 1 & 5 & 3 \\ 0 & 2 & 4 & 8 \end{bmatrix}$$

왼쪽 위 영역은 다음과 같다.

$$\begin{bmatrix} 1 & 3 \\ 4 & 6 \end{bmatrix}$$

최댓값은 6 이다.

오른쪽 위 영역은 다음과 같다.

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

최댓값은 2 이다.

전체 출력은 다음과 같다.

$$\begin{bmatrix} 6 & 2 \\ 2 & 8 \end{bmatrix}$$

13.3 평균 풀링

Average Pooling 은 지정된 영역 안의 값들의 평균을 계산한다.

$$\begin{bmatrix} 1 & 3 \\ 4 & 6 \end{bmatrix}$$

평균값은 다음과 같다.

$$\frac{1 + 3 + 4 + 6}{4} = 3.5$$

최대 풀링이 가장 강한 특징을 유지한다면 평균 풀링은 영역 전체의 정보를 부드럽게 요약한다.

13.4 글로벌 평균 풀링

Global Average Pooling 은 각 채널의 모든 공간 값을 평균하여 하나의 값으로 변환한다.

입력: $7 \times 7 \times 512$

↓

출력: $1 \times 1 \times 512$

글로벌 평균 풀링의 장점은 다음과 같다.

- 완전연결 계층의 파라미터 수 감소
- 과적합 완화
- 채널별 특징 직접 요약
- 입력 크기에 대한 유연성 향상

현대 CNN에서는 평탄화와 큰 완전연결 계층 대신 글로벌 평균 풀링을 많이 사용한다.

14. 평탄화와 완전연결 계층

14.1 평탄화

합성곱 계층의 출력은 일반적으로 3 차원 특성 맵이다.

예를 들어 출력 크기가 다음과 같다고 하자.

$$7 \times 7 \times 64$$

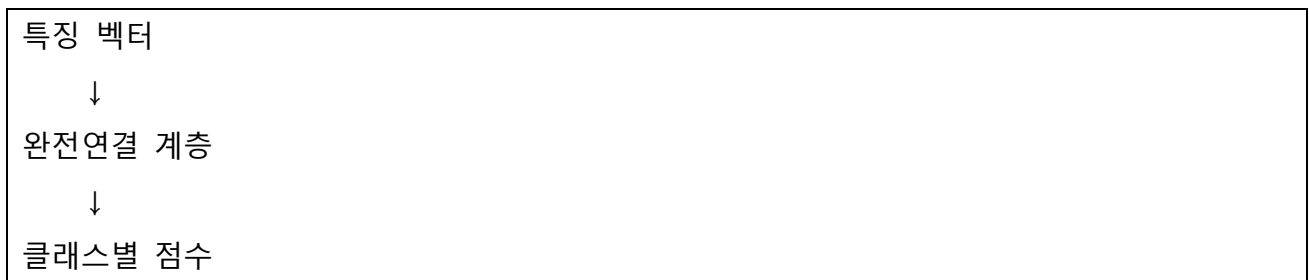
평탄화를 적용하면 다음 길이의 1 차원 벡터가 된다.

$$7 \times 7 \times 64 = 3,136$$

평탄화는 데이터 형태만 변경하며 학습 파라미터는 없다.

14.2 완전연결 계층

완전연결 계층은 CNN 이 추출한 특징을 종합하여 최종 분류를 수행한다.



예를 들어 10 개 클래스를 분류한다면 마지막 계층은 10 개의 값을 출력한다.

이 값을 로짓(Logit)이라고 한다.

15. Softmax 와 클래스 확률

Softmax 는 로짓을 합이 1 인 확률 분포로 변환한다.

클래스 (i)의 확률은 다음과 같다.

$$P(y = i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

예를 들어 출력 결과가 다음과 같을 수 있다.

클래스	확률
고양이	0.72
강아지	0.18
토끼	0.06
기타	0.04

가장 높은 확률을 가진 클래스가 최종 예측 결과가 된다.

PyTorch 의 CrossEntropyLoss 는 내부적으로 로그 Softmax 와 손실 계산을 함께 처리한다. 따라서 학습 모델의 마지막 계층에서 Softmax 를 직접 적용하지 않는 것이 일반적이다.

16. CNN 의 순전파

순전파(Forward Propagation)는 입력 이미지가 모델을 통과하여 예측값을 생성하는 과정이다.

1. 이미지 입력
2. 합성곱 수행
3. 활성화 함수 적용
4. 풀링 수행
5. 합성곱 블록 반복
6. 특성 맵을 벡터로 변환
7. 완전연결 계층 계산
8. 클래스 로짓 출력
9. 손실 계산

하나의 합성곱 블록은 다음처럼 표현할 수 있다.

$$Z^{(1)} = X * K^{(1)} + b^{(1)}$$

$$A^{(1)} = \text{ReLU}(Z^{(1)})$$

$$P^{(1)} = \text{Pooling}(A^{(1)})$$

최종 예측값은 다음과 같이 계산된다.

$$\hat{y} = f(P^{(1)}; W, b)$$

17. 손실 함수

다중 클래스 분류에서는 일반적으로 교차 엔트로피 손실을 사용한다.

정답 클래스가 (y)이고 모델이 정답 클래스에 부여한 확률이 (p_y)라면 손실은 다음과 같다.

$$L = -\log(p_y)$$

정답 클래스의 예측 확률이 높을수록 손실은 작아진다.

- 정답 확률 0.9 → 손실이 작음
- 정답 확률 0.5 → 손실이 중간
- 정답 확률 0.1 → 손실이 큼

학습의 목적은 전체 데이터에 대한 손실을 최소화하는 것이다.

18. CNN 의 역전파

18.1 역전파의 개념

역전파(Backpropagation)는 예측 오류가 각 파라미터에 얼마나 영향을 받았는지 기울기를 계산하는 과정이다.

순전파와 반대 방향으로 진행된다.

손실
↓
출력 계층
↓
완전연결 계층
↓
풀링 계층
↓
활성화 계층
↓
합성곱 계층

계산된 기울기를 이용하여 합성곱 커널과 완전연결 계층의 가중치를 수정한다.

18.2 커널 가중치 업데이트

커널은 다음과 같이 업데이트된다.

$$K \leftarrow K - \eta \frac{\partial L}{\partial K}$$

각 기호의 의미는 다음과 같다.

- K : 현재 커널 가중치
- η : 학습률
- L : 손실 함수
- $\frac{\partial L}{\partial K}$: 커널에 대한 손실의 기울기

학습이 반복되면서 커널은 손실을 줄이는 방향으로 변경된다.

18.3 ReLU 의 역전파

ReLU 의 미분은 다음과 같다.

$$\text{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

순전파에서 입력이 양수였던 위치에는 기울기가 전달되고, 입력이 0 이하였던 위치에는 기울기가 전달되지 않는다.

18.4 최대 풀링의 역전파

최대 풀링에서는 순전파 시 최댓값으로 선택된 위치에만 기울기가 전달된다.
다음 영역에서 최댓값은 6 이다.

$$\begin{bmatrix} 1 & 3 \\ 4 & 6 \end{bmatrix}$$

상위 계층에서 전달된 기울기가 2 라면 다음처럼 전달된다.

$$\begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}$$

최댓값이 아니었던 위치는 순전파 출력에 직접 사용되지 않았기 때문에 기울기가 0 이다.

19. 간단한 PyTorch CNN 구현

```
import torch
import torch.nn as nn

class SimpleCNN(nn.Module):
    """
    28×28 크기의 흑백 이미지를 입력받아
    10 개 클래스 중 하나로 분류하는 CNN 모델이다.
    """
```

```

def __init__(self) -> None:
    """
    CNN의 특징 추출 계층과 분류 계층을 정의한다.
    """

    # nn.Module 부모 클래스의 초기화 메서드를 호출한다.
    # PyTorch가 모델의 계층과 파라미터를 관리하려면 반드시 필요하다.
    super().__init__()

    # 이미지에서 시각 특징을 추출하는 계층을 정의한다.
    self.features = nn.Sequential(
        # 첫 번째 합성곱 계층이다.
        # 입력 채널 1개를 32개의 출력 채널로 변환한다.
        # 3x3 커널과 패딩 1을 사용하므로 공간 크기는 유지된다.
        # 입력: [배치, 1, 28, 28]
        # 출력: [배치, 32, 28, 28]
        nn.Conv2d(
            in_channels=1,
            out_channels=32,
            kernel_size=3,
            stride=1,
            padding=1,
        ),

        # 합성곱 결과에 비선형성을 추가한다.
        # 음수는 0으로 바꾸고 양수는 그대로 유지한다.
        nn.ReLU(),

        # 2x2 영역에서 최댓값을 선택한다.
        # 가로와 세로 크기를 절반으로 줄인다.
        # 입력: [배치, 32, 28, 28]
        # 출력: [배치, 32, 14, 14]
        nn.MaxPool2d(

```

```

        kernel_size=2,
        stride=2,
    ),

    # 두 번째 합성곱 계층이다.
    # 32 개의 입력 채널을 64 개의 출력 채널로 변환한다.
    # 입력: [배치, 32, 14, 14]
    # 출력: [배치, 64, 14, 14]
    nn.Conv2d(
        in_channels=32,
        out_channels=64,
        kernel_size=3,
        stride=1,
        padding=1,
    ),

    # 두 번째 합성곱 결과에 비선형성을 추가한다.
    nn.ReLU(),

    # 두 번째 최대 풀링을 수행한다.
    # 입력: [배치, 64, 14, 14]
    # 출력: [배치, 64, 7, 7]
    nn.MaxPool2d(
        kernel_size=2,
        stride=2,
    ),
)

# 추출한 특징을 사용하여 클래스를 분류하는 계층을 정의한다.
self.classifier = nn.Sequential(
    # [배치, 64, 7, 7] 형태를
    # [배치, 64×7×7] 형태로 변환한다.
    nn.Flatten(),

```

```

# 3,136 개의 입력 특징을 128 개의 은닉 특징으로 변환한다.
nn.Linear(
    in_features=64 * 7 * 7,
    out_features=128,
),

# 완전연결 계층의 출력에 비선형성을 추가한다.
nn.ReLU(),

# 학습 중 50%의 뉴런을 무작위로 비활성화한다.
# 특정 특징에 지나치게 의존하는 현상을 줄인다.
nn.Dropout(p=0.5),

# 128 개의 특징을 10 개 클래스의 로짓으로 변환한다.
# CrossEntropyLoss 가 내부적으로 Softmax 관련 계산을 수행하므로
# 모델 내부에서는 Softmax 를 적용하지 않는다.
nn.Linear(
    in_features=128,
    out_features=10,
),
)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    입력 이미지가 CNN 을 통과하는 순전파 과정을 정의한다.

    Args:
        x: [배치, 1, 28, 28] 형태의 이미지 텐서

    Returns:
        [배치, 10] 형태의 클래스별 로짓
    """

```

```

        # 합성곱, ReLU, 풀링 계층을 통과시켜 특징을 추출한다.
        x = self.features(x)

        # 추출된 특징을 평탄화하고 완전연결 계층에 전달한다.
        x = self.classifier(x)

        # 클래스별 로짓을 반환한다.
        return x

# CNN 모델 객체를 생성한다.
model = SimpleCNN()

# 배치 크기 8, 채널 1, 높이 28, 너비 28 인
# 임의의 흑백 이미지 텐서를 생성한다.
sample_images = torch.randn(8, 1, 28, 28)

# 모델에 이미지를 전달하여 순전파를 실행한다.
sample_outputs = model(sample_images)

# 이미지 8 장 각각에 대해 10 개의 클래스 점수가 생성된다.
print(sample_outputs.shape)
실행 결과:
torch.Size([8, 10])

```

20. CNN 학습 코드의 핵심 흐름

```

# 이전 미니배치에서 계산된 기울기를 초기화한다.
# PyTorch 는 기본적으로 기울기를 누적하므로 반복마다 초기화해야 한다.
optimizer.zero_grad()

# 입력 이미지를 CNN 에 전달하여 클래스별 로짓을 계산한다.

```

```
outputs = model(images)

# 모델 출력과 실제 정답 레이블을 비교하여 손실을 계산한다.
loss = criterion(outputs, labels)

# 손실을 기준으로 각 파라미터의 기울기를 계산한다.
loss.backward()

# 계산된 기울기를 이용하여 모델 파라미터를 갱신한다.
optimizer.step()
```

전체 학습 흐름은 다음과 같다.

```
미니배치 입력
    ↓
순전파
    ↓
예측값 생성
    ↓
손실 계산
    ↓
기울기 초기화
    ↓
역전파
    ↓
파라미터 업데이트
    ↓
다음 미니배치
```

21. 대표적인 CNN 구조

21.1 LeNet

LeNet 은 손글씨 숫자 인식을 위해 개발된 초기 CNN 구조이다.

입력

- 합성곱
- 풀링
- 합성곱
- 풀링
- 완전연결
- 출력

CNN 의 기본 구조를 이해하는 데 적합하다.

21.2 AlexNet

AlexNet 은 대규모 이미지 분류에서 깊은 CNN 의 성능을 입증한 모델이다.

주요 특징은 다음과 같다.

- 여러 합성곱 계층 사용
- ReLU 사용
- GPU 기반 학습
- 드롭아웃 적용
- 데이터 증강 사용

21.3 VGG

VGG 는 3×3 합성곱을 반복적으로 쌓은 규칙적인 구조이다.

3×3 합성곱

- 3×3 합성곱
- 풀링
- 반복

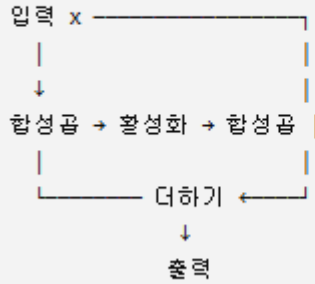
구조가 단순하고 이해하기 쉽지만 파라미터 수가 많고 연산량이 크다는 단점이 있다.

21.4 ResNet

네트워크가 매우 깊어지면 기울기 소실과 학습 성능 저하 문제가 발생할 수 있다.
ResNet 은 입력을 몇 개 계층을 건너뛰어 출력에 직접 더하는 잔차 연결을 사용한다.

$$y = F(x) + x$$

구조는 다음과 같다.



잔차 연결은 기울기가 더 직접적으로 전달될 수 있도록 하여 깊은 네트워크의 학습을 돕는다.

21.5 MobileNet

MobileNet 은 모바일과 임베디드 환경에서 효율적으로 실행하도록 설계된 경량 CNN 이다.
일반 합성곱을 다음 두 연산으로 분리한다.

Depthwise Convolution



Pointwise Convolution

이를 Depthwise Separable Convolution 이라고 한다.

일반 합성곱보다 파라미터 수와 계산량을 크게 줄일 수 있다.

22. 핵심 용어 정리

용어	설명
CNN	합성곱 연산을 기반으로 특징을 추출하는 신경망
커널	입력 위를 이동하며 특징을 검출하는 작은 가중치 행렬
특성 맵	커널 적용 결과로 생성된 출력 데이터
채널	이미지나 특성 맵의 깊이 방향 정보
스트라이드	커널이 한 번에 이동하는 칸 수
패딩	입력 가장자리에 값을 추가하는 처리
ReLU	음수를 0 으로 변환하는 활성화 함수
풀링	특성 맵의 공간 크기를 축소하는 연산
평탄화	다차원 특성 맵을 1 차원으로 변환하는 연산
로짓	출력 계층이 생성한 클래스별 원시 점수
수용 영역	한 뉴런이 입력에서 영향을 받는 영역
역전파	손실을 이용해 파라미터의 기울기를 계산하는 과정
가중치 공유	동일 커널을 입력의 모든 위치에 적용하는 방식

3. STT (Speech-to-Text)

1. 개요

사운드를 텍스트로 변환하는 기술(STT, Speech-to-Text) 또는 ****자동 음성 인식(ASR, Automatic Speech Recognition)****은 사람의 음성을 컴퓨터가 이해할 수 있는 문자(Text)로 변환하는 인공지능 기술입니다.

예를 들어 사용자가 마이크에 다음과 같이 말하면,

"안녕하세요. 오늘 날씨를 알려주세요."

STT 시스템은 이를 다음과 같은 텍스트로 변환합니다.

안녕하세요. 오늘 날씨를 알려주세요.

오늘날 STT 기술은 AI 비서, 콜센터, 회의록 작성, 자막 생성, 스마트폰 음성 입력 등 다양한 분야에서 활용되고 있습니다.

2. 등장 배경

초기의 컴퓨터는 키보드와 마우스를 이용한 입력만 가능했습니다.

그러나 모바일 기기와 AI 기술이 발전하면서 사람들은 더욱 자연스러운 방식으로 컴퓨터와 대화하기를 원하게 되었습니다.

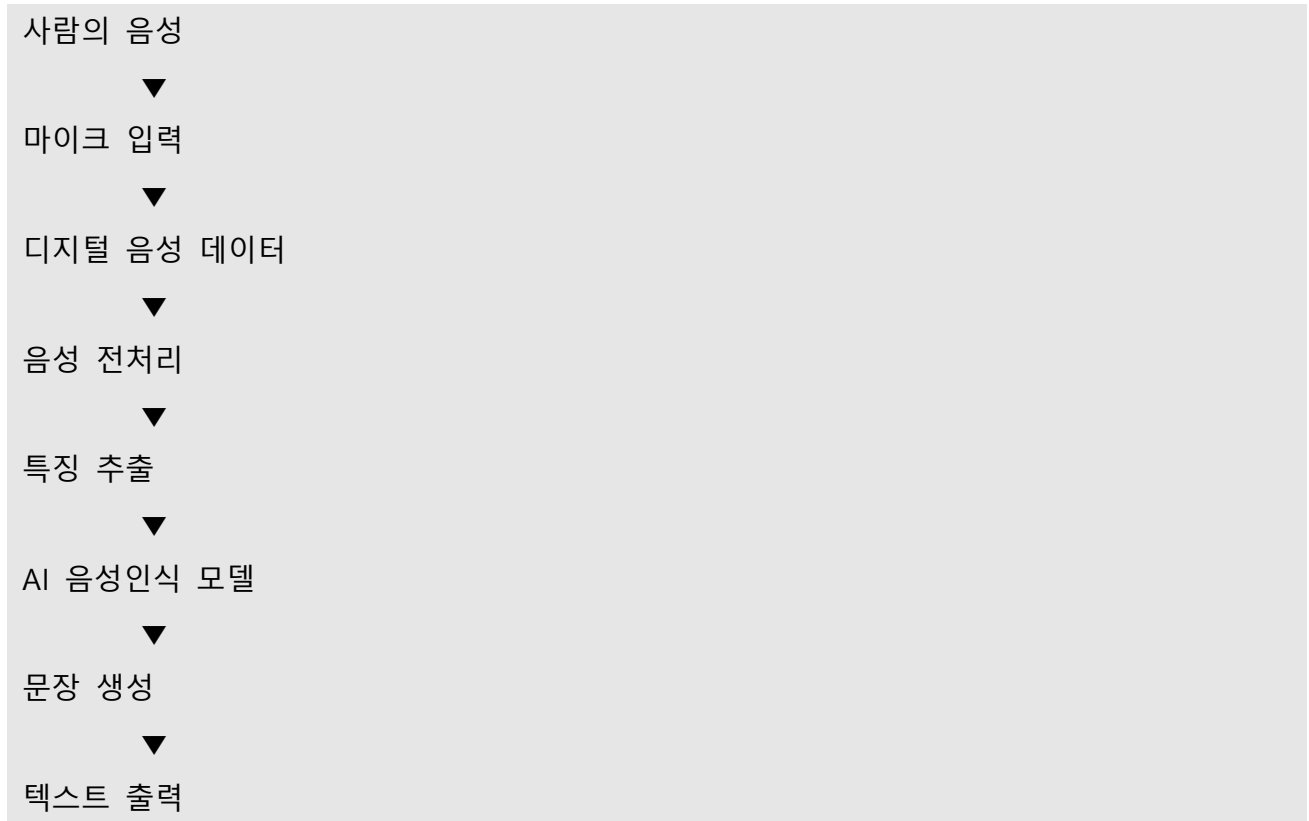
대표적인 요구사항은 다음과 같습니다.

- 키보드 없이 문서 작성
- 자동차 운전 중 음성 명령
- 회의 자동 기록
- 실시간 자막
- AI 챗봇과 음성 대화

이러한 요구를 해결하기 위해 음성을 문자로 변환하는 STT 기술이 발전하게 되었습니다.

3. STT의 기본 개념

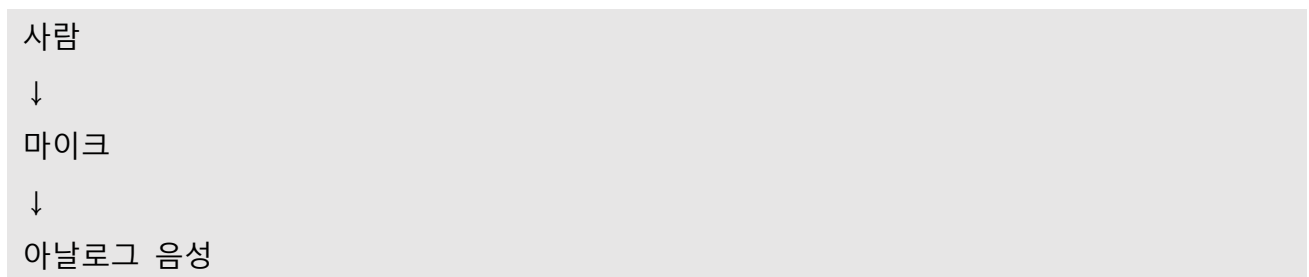
STT는 크게 다음과 같은 과정을 수행합니다.



4. STT 처리 과정

STEP 1. 음성 입력(Audio Capture)

가장 먼저 사람의 음성을 마이크로 입력받습니다.



이때 음성은 아직 컴퓨터가 이해할 수 없습니다.

STEP 2. 디지털 변환(Audio Digitization)

컴퓨터는 아날로그 음성을 디지털 신호로 변환합니다.
이를 **ADC(Analog-to-Digital Converter)**라고 합니다.

대표적인 설정은

- Sampling Rate : 16000Hz
- Mono
- PCM 16bit

입니다.

예)

0.12
0.18
0.20
0.15
...

이 숫자들이 음성 파형을 표현합니다.

STEP 3. 음성 전처리

녹음된 음성에는 잡음이 포함될 수 있습니다.
따라서 다음과 같은 전처리를 수행합니다.

- Noise Reduction
- Echo Cancellation
- Voice Activity Detection(VAD)
- Silence Removal
- Volume Normalization

예)

원본 : -----안녕하세요-----
↓
무음 제거 : 안녕하세요

STEP 4. 특징 추출(Feature Extraction)

AI는 원시 음성 데이터를 그대로 학습하지 않습니다.

음성의 중요한 특징만 추출합니다.

대표적인 특징은 **MFCC** Mel Frequency Cepstral Coefficients

사람의 청각 특성을 반영한 대표적인 음성 특징입니다.

또는

- Spectrogram
- Mel Spectrogram
- Log-Mel Spectrogram

등을 사용합니다.

예)

음성

↓

Mel Spectrogram

↓

AI 입력

5. AI 음성 인식 모델

최근 STT는 대부분 딥러닝 기반입니다.

대표적인 모델은

RNN

초기의 음성인식

음성

↓

RNN

↓

문장

LSTM

긴 문장을 처리하기 위해 RNN을 개선한 모델입니다.

Transformer

현재 가장 많이 사용하는 구조입니다.

장거리 문맥을 매우 잘 이해합니다.

Whisper

OpenAI에서 공개한 대표적인 음성 인식 모델입니다.

특징

- 다국어 지원
- 높은 정확도
- 노이즈에 강함
- 번역 지원

Faster-Whisper

Whisper를 CTranslate2 기반으로 최적화한 구현입니다.

장점

- CPU 속도 향상
- GPU 속도 향상
- 메모리 절약
- int8 추론 지원

PyCharm 프로젝트에서도 많이 사용됩니다.

6. STT의 전체 처리 구조

사람의 음성
↓
마이크
↓
PCM WAV 생성
↓
Noise Reduction
↓
VAD
↓
Feature Extraction
↓
Transformer
↓
Language Model
↓
Text Decoder
↓
최종 텍스트

7. Language Model의 역할

음성만 듣고 문장을 만드는 것이 아니라, 언어 모델이 문맥을 고려합니다.

예)

음성

오늘은 날씨가...

가능한 결과

오늘은 날씨가 좋습니다.
오늘는 날씨가 좋습니다.

언어 모델은 첫 번째 문장을 선택합니다.

8. Decoder의 역할

AI가 예측한 토큰을 사람이 읽을 수 있는 문장으로 변환합니다.

예)

[오늘]
↓
[은]
↓
[날]
↓
[씨]
↓
[가]
↓
좋습니다
↓
오늘은 날씨가 좋습니다.

9. 음성 인식에서 사용하는 주요 기술

기술	설명
Voice Activity Detection(VAD)	음성이 있는 구간만 검출
Noise Reduction	잡음 제거
Echo Cancellation	메아리 제거
Beam Search	가장 가능성이 높은 문장 탐색
Language Detection	언어 자동 감지
Timestamp	단어별 시간 계산
Tokenization	문장을 토큰으로 분리
Transformer	문맥 이해
Decoder	최종 문장 생성

10. 최신 STT 기술

최근에는 단순한 음성 인식을 넘어 다음 기능까지 제공합니다.

Speaker Diarization

누가 말했는지 구분

A :
안녕하세요.

B :
반갑습니다.

Translation

음성을 다른 언어로 번역

한국어
↓
영어

Real-time STT

실시간 자막 생성

예)

- Zoom
- Teams
- YouTube Live

Streaming STT

말하는 동시에 텍스트 생성

안녕하세...
↓
안녕하세요.

Word Timestamp

단어마다 시간을 제공합니다.

0.20
안녕하세요
0.75
오늘은
1.15
날씨가

11. 대표적인 STT 프레임워크

프레임워크	특징
Whisper	정확도가 높고 다국어 지원
Faster-Whisper	Whisper를 고속화한 구현
Vosk	오프라인 지원
NVIDIA NeMo	대규모 음성 AI 연구용
SpeechBrain	PyTorch 기반 음성 연구 프레임워크
Wav2Vec 2.0	자기지도학습 기반 음성 모델
DeepSpeech	Mozilla 음성 인식 모델

12. 활용 분야

- 스마트폰 음성 입력
- AI 비서
- 음성 검색
- 회의록 자동 작성
- 실시간 자막
- 고객센터 상담 분석
- 의료 음성 기록
- 법률 녹취록 작성

- 방송 자막 생성
- 유튜브 자동 자막
- 온라인 강의 자막
- 음성 기반 챗봇
- 자동차 음성 제어
- 스마트홈 음성 제어

13. 장점

- 키보드 없이 입력 가능
- 빠른 문서 작성
- 높은 생산성
- 장애인 접근성 향상
- 실시간 회의록 작성
- AI 음성 인터페이스 구현 가능

14. 한계점

- 주변 소음이 많으면 정확도가 저하될 수 있습니다.
- 여러 사람이 동시에 말하면 인식이 어려울 수 있습니다.
- 전문 용어나 고유명사는 오인식될 수 있습니다.
- 방언, 억양, 발음 차이에 따라 성능이 달라질 수 있습니다.
- 실시간 처리에서는 지연 시간(latency)과 연산 자원 관리가 중요합니다.

15. 전체 동작 흐름 요약

사용자 음성



마이크 입력



아날로그 → 디지털 변환



음성 전처리 (잡음 제거, VAD)



특징 추출 (MFCC, Mel Spectrogram)



딥러닝 음성 인식 모델 (Transformer, Whisper 등)



언어 모델 기반 문맥 보정



디코더



최종 텍스트 생성

정리

STT(Speech-to-Text)는 음성을 디지털 신호로 변환한 뒤 전처리와 특징 추출을 거쳐, 딥러닝 기반 음성 인식 모델이 언어적 문맥을 고려하여 텍스트를 생성하는 기술입니다. 최근에는 Transformer와 Whisper 계열 모델의 발전으로 다국어 지원, 실시간 스트리밍, 화자 구분, 번역 등 다양한 기능을 제공하며, AI 비서, 회의록 작성, 자막 생성, 음성 검색 등 여러 응용 분야의 핵심 기술로 활용되고 있습니다.